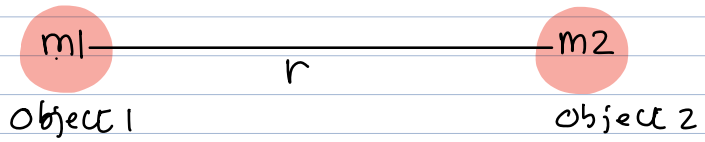
Compute Gravitational.



$$F = G \frac{m1 \cdot m2}{r^2}$$

G = Universal Constant of Gravitation = 6.67×10^{-11}

01-compute-gravitational.c

```
#include <stdio.h>

int main(void)
{
    double G = 6.67e-11;
    double m1, m2, r, F;

    printf("Enter mass of object 1 in kg:");
    scanf("%lf", &m1);

    printf("Enter mass of object 2 in kg:");
    scanf("%lf", &m2);

    printf("Enter the distance between them in meter:");
    scanf("lf", &r);

    F = (G * m1 * m2)/(r*r);
    printf("Gravitational force of attraction is %lf Newton\n", F);

    return (0);
}
```

m1 = mass of earth
m2 = mass of sun
r = earth-sun distance

Earth - Jupitor	Sun - Saturn.	Venus - Mercury.
m_{Earth}	m_{Sun}	m_{venus}
$m_{Jupitor}$	m_{saturn}	$m_{mercury}$
$r_{Earth-Jupitor}$	$r_{Sun-saturn}$	$r_{venus-mercury}$

```
double m_earth, m_jupitor, r_earth_jupitor;
double m_Sun, m_saturn, r_Sun_saturn;
double m_venus, m_mercury, r_venus_mercury;
```

```
m1 : double
m2 : double
r : double
```
Block

variable_name.

```
variable_name.m1
variable_name.m2
variable_name.r
```

```
Struct ComputeGravitational
{
    double m1; // member
    double m2; // member
    double r; // member
};

Struct ComputeGravitational earthSun;
```

```
earthSun.m1 = earth
earthSun.m2 = Sun
earthSun.r
```

```
F = (G * earthSun.m1 * earthSun.m2) /
    (earthSun.r * earthSun.r);
```

Paradigm = एखादी गोष्ट करण्याची अनुभवसिद्ध पद्धत

= A way of doing certain thing born out of experience.

Programming Paradigm =

- Recommended ways of doing programming, born out of experience
- Programming languages support one or more paradigms.
- Support == Provide statements which facilitate programming according to a paradigm.

C++ 98:

1) Procedural Programming (borrowed from C)
2) Modular Programming
3) Object based Programming
4) Object oriented programming
5) Generic / Type Generic Programming

C++ 11:

6) Functional Programming.

C++ 20: [Meta-Programming] [template Meta-Programming]

C++ 20 : Coroutine (Concurrent Programming)

C++ 23 : [Aspect Oriented Programming] [Reflection]

Procedural Programming :

[1] Divide the task into multiple sub-routines.

[2] You can recursively divide sub-routines until they become simple enough to implement.

[3] Choose the best algorithm for each sub-routine
[Best Algo : Good trade-off space / time]

[4] Write a driver routine (= main() function) to combine all sub-routines.

No focus on data !!

Each function accepts data with pre-condition and promises to output data with post-condition

[1] Built-in data types [2] Built-in operators. [3] Assignment stmt.
[4] Branching / looping stmts [5] functions. [6] Array
[7] structure [8] Pointer Adv C 의 장점

C++ features: 1) Reference variables
2) function with default params.
3) function overloading
↳ [Name Mangling]
